

LISTA DE EXERCÍCIOS 5.3

GABARITO...

OBS. Resoluções em sala de aula podem englobar mais pontos de vista e atributos do que apenas as respostas simplificadas apresentadas no gabarito.

1. Uma forma de lidar com conversão de parâmetros em sistemas RPC é ter cada máquina enviando parâmetros em sua representação nativa, com a outra realizando a tradução, se necessário. O sistema nativo poderia ser indicado por um código no primeiro byte. Entretanto, como localizar o primeiro byte na primeira palavra é precisamente o problema, isso poderia funcionar?

R. Considere que, quando um computador envia um byte 0, ele sempre chega no byte 0 (independente da ordenação). Assim, o computador destino pode simplesmente acessar o byte 0 (usando uma instrução para byte) e o código estará lá. Não importa se este é um byte menos significativo ou mais significativo (a representação bit a bit terá que ser tratada também após a localização do byte). Um esquema alternativo é colocar o código em todos os bytes da primeira palavra. Assim, não importa qual byte é examinado, o código estará lá.

2. O C possui um construto de linguagem chamado *union* o qual para a implementação de um registro pode manter vários valores. Em tempo de execução, não existe forma certa de dizer qual valor está na *union* em um dado momento. Esta característica do C possui qualquer implicação para RPC? Explique sua resposta.

R. Se o sistema *runtime* não pode dizer que tipo de valor está no campo, ele não pode realizar o *marshalling* corretamente. Assim, *unions* não podem ser toleradas em um sistema RPC a menos que exista um campo de tag que diga de forma não ambígua que valor o campo variável possui. A *tag* não deve estar sob controle do usuário.

3. Ao invés de deixar um servidor se registrar em um *daemon*, como é feito com o DCE, também poderíamos sempre escolher uma mesma porta. Esta porta poderia então, ser usada em referências a objetos no mesmo espaço de endereçamento do servidor. Qual é a principal desvantagem desta abordagem?

R. A principal desvantagem é a dificuldade em se alocar objetos dinamicamente em servidores. Além disso, muitas portas precisam ser fixadas, ao invés de apenas uma (i.e. a porta do daemon). Para máquinas possivelmente possuindo um grande número de servidores, uma atribuição estática de portas não é uma boa ideia.

4. Quando pensamos em uso de middleware em sistemas distribuídos, para a distribuição de tarefas, qual é a complexidade envolvida considerando passagem de parâmetros por valor e referência?

R. Passagem de parâmetro por valor não implicaria no uso de uma representação intermediária, de modo que o destinatário possa representar o dado transmitido em sua plataforma, sem problemas de conversão para tipos de dados. A passagem de valor por referência lida com referência a posições de memória. Desta forma, criar uma referência remota a uma posição de memória local seria problemático. Este problema pode

ser superado por meio de uma política copy/restore onde a ideia é “replicar” o conteúdo relativo a uma posição de memória no destinatário (sem referência a uma posição de memória real), realizar as operações desejadas e posteriormente realizar quaisquer modificações necessárias no sistema local (algo similar a uma política de coerência de cache ↔ memória principal).